

Git y Gitlab

Control de versiones

- Gestión de ficheros a lo largo del tiempo
 - Evolución del trabajo
- Gestión del versionado de los ficheros
 - Si un archivo se corrompe o hemos cometido un fallo volvemos atrás
- Mecanismo para compartir ficheros
- Habitualmente tenemos nuestro propio mecanismo y modelo de trabajo
 - Versionado: Documento.docx, Documento_v2.docx
 - Herramientas: Dropbox, Adjunto correo.

Control de versiones

- Las metodologías/mecanismos particulares no escalan para proyectos de desarrollo
- Un SCV permite
 - Crear copias de seguridad y restaurarlas
 - Sincronizar (mantener al día) a los desarrolladores respecto a la última versión de desarrollo
 - Deshacer cambios
 - Tanto problemas puntuales, como problemas introducidos hace tiempo
 - Gestionar la autoría del código – Realizar pruebas (aisladas)
 - Simples o utilizando el mecanismo de branches/merges

Tipos de SCV

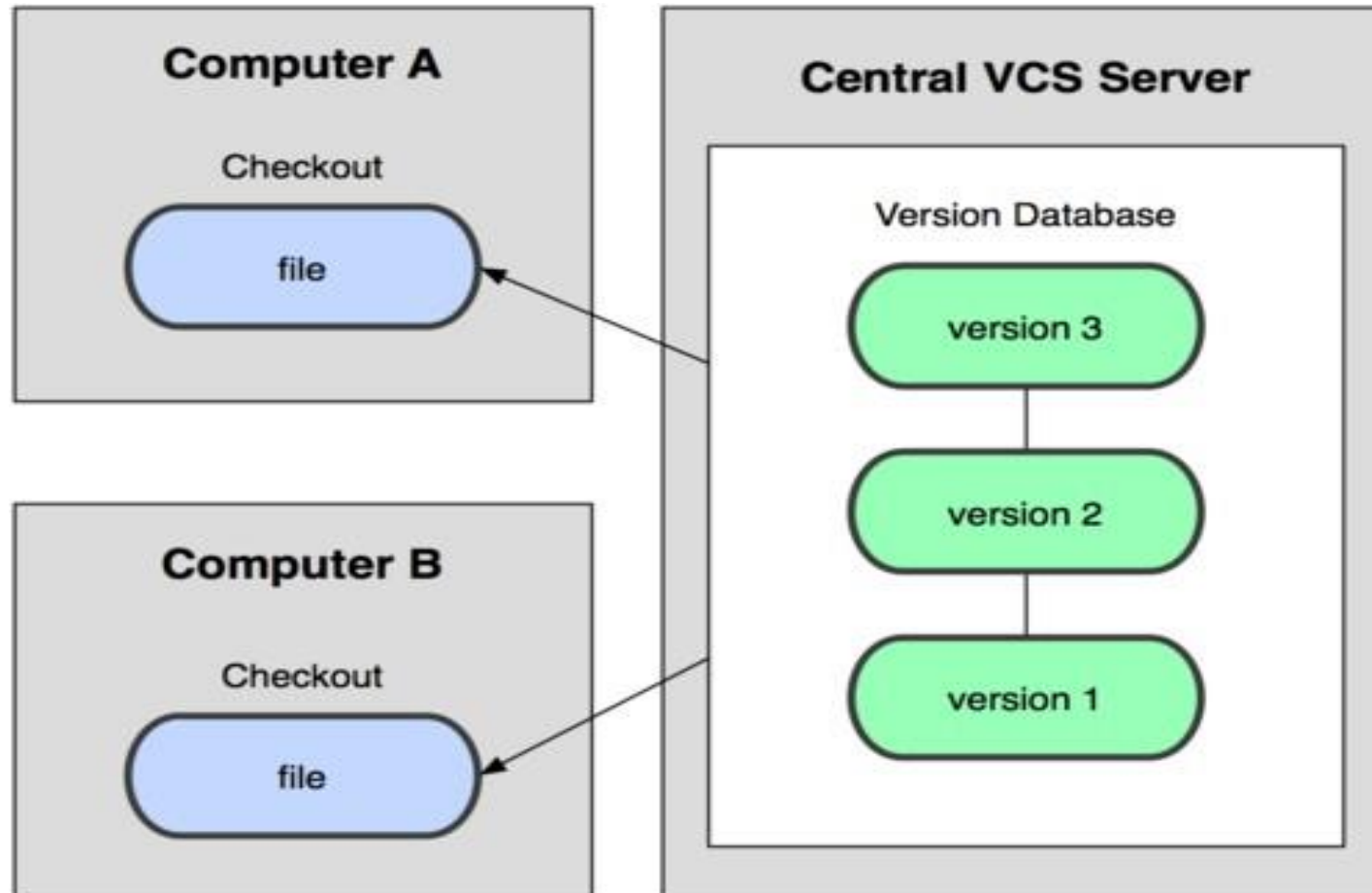
- **Centralizados**

- Existe un servidor centralizado que almacena el repositorio completo
- La comunicación/colaboración entre desarrolladores se lleva a cabo (forzosamente) utilizando el repositorio centralizado
- Son más simples de usar
- Los modelos de trabajo son más restringidos

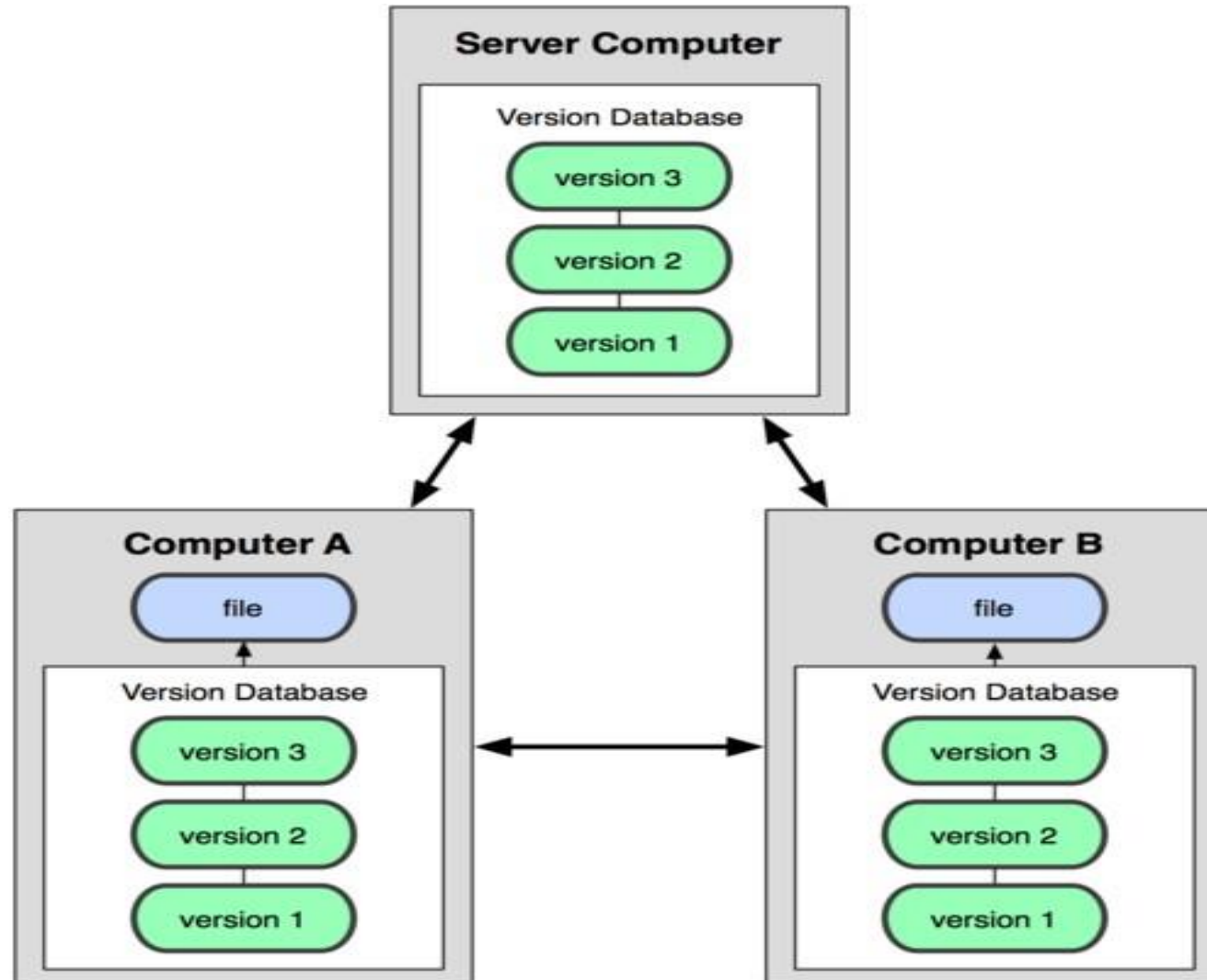
- **Distribuidos**

- Cada desarrollador contiene una copia completa de todo el repositorio
- Los mecanismos de comunicación/colaboración entre desarrolladores son más abiertos
- Son (un poco) más difíciles de utilizar que los sistemas centralizados
- Los modelos de trabajo son más flexibles
- “Los branches/merges son más simples”

SCV Centralizado



SCV Distribuido



Algunas herramientas

- **Centralizados**

- Subversion (SVN)
 - <http://subversion.apache.org>, <http://subversion.tigris.org/>
- Concurrent Version System (CVS)
 - <http://www.nonfnu.org/cvs/>
- Microsoft Visual Source Safe
- Perforce

- **Distribuidos**

- Git
 - <http://git-scm.com>
- Mercurial
 - <http://hg-scm.com>
- Bazaar, DARCS

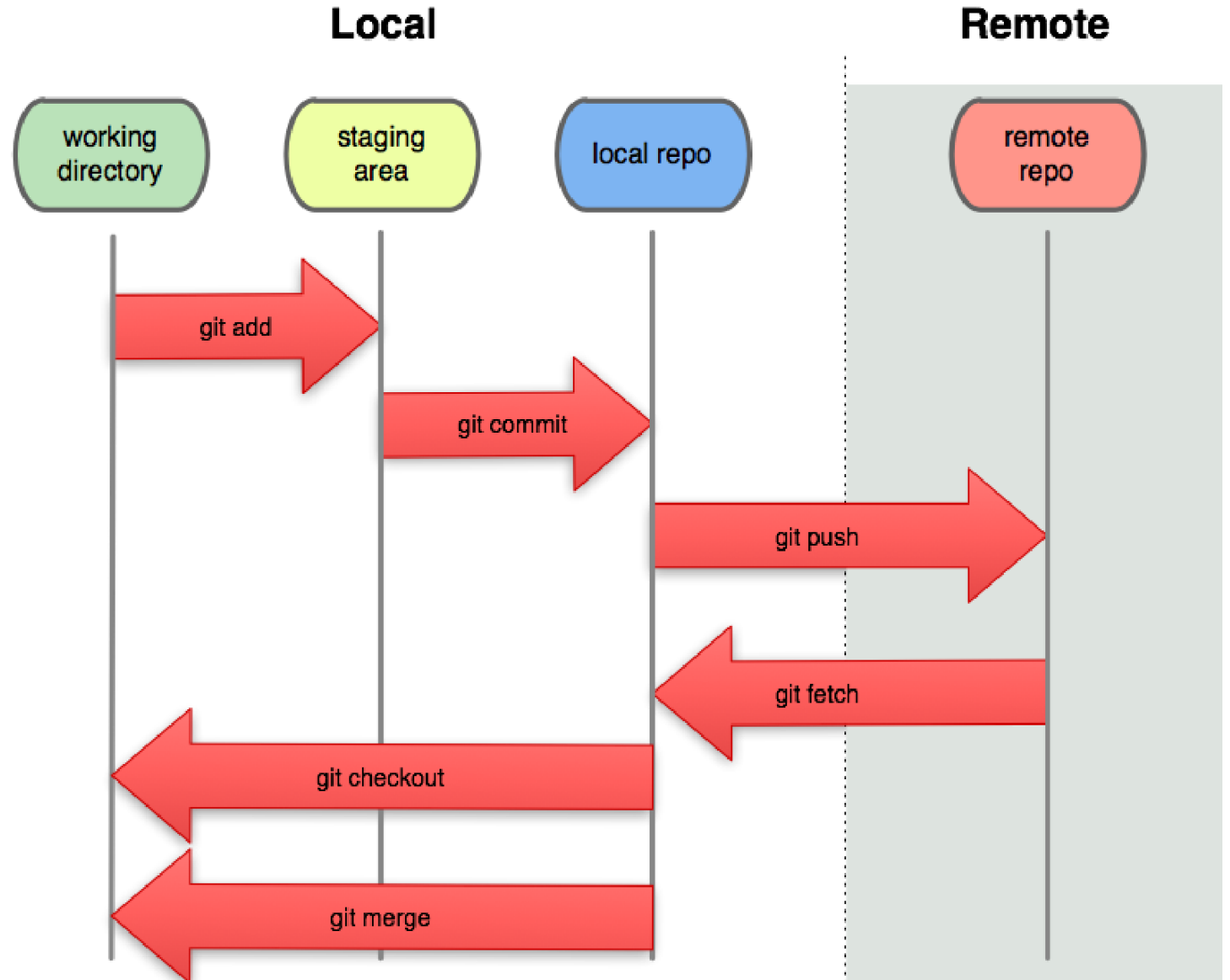
Un sistema para el control de versiones: Git

- Fue creado por Linus Torvalds, el creador de Linux, en el año 2005
- Los objetivos de Git:
 - Velocidad
 - Soporte a desarrollos no lineales
 - Distribuido
 - Capaz de gestionar grandes proyectos



Por qué Git

- Todo es local
 - Operaciones más rápidas
 - Puedes trabajar sin red
 - Todos los repositorios de los desarrolladores son iguales
 - En caso de emergencia puede servir de backup



Instalación de GIT

- Básico
 - Linux: `sudo apt-get/yum install git (git-cola, git-meld)`
 - [Windows](#) ó instalar Cygwin+git, git bash
- Entornos gráficos
 - [SourceTree](#) (Windows / MacOSX) (Gratuita)
 - Smartgit (Multiplataforma) (free non-commercial)
 - MacOSX: [GitX](#)
 - Windows: [TortoiseGIT](#)
- Integrado en IDEs
 - Eclipse
 - IntelliJ
 - Integrada en Visual Studio Code (necesita plugin)

Configurando GIT

- Consulta
 - `git config --list`
- Modificación por proyecto o global (también por sistema)
 - `git config --global user.name "Bugs Bunny"`
 - `git config --global user.email bugs@gmail.com`
- Con el comando “`git config --list`” se puede comprobar el estado.

Crear un repositorio

- Creamos un directorio de trabajo:
 - `mkdir trabajodir`
 - `cd trabajodir`
- Editamos algún fichero
 - `vi unfichero.txt`
 - (se puede utilizar cualquier editor)
- Observamos que solo está el fichero
 - `ls -la`
- Creamos un repositorio
 - `git init`
- Ha aparecido un directorio oculto (.git)
 - `ls -la`
- Dentro se alojan todos archivos y carpetas internos que gestionan un repositorio de git

Directorio de trabajo

- Es el directorio en el que editamos nuestros ficheros
- No todos los ficheros de ese directorio son útiles
 - puede haber ficheros de cache o resultados de ejecuciones
- En el ejemplo anterior, el directorio lo hemos llamado “trabajodir”

Staging Area

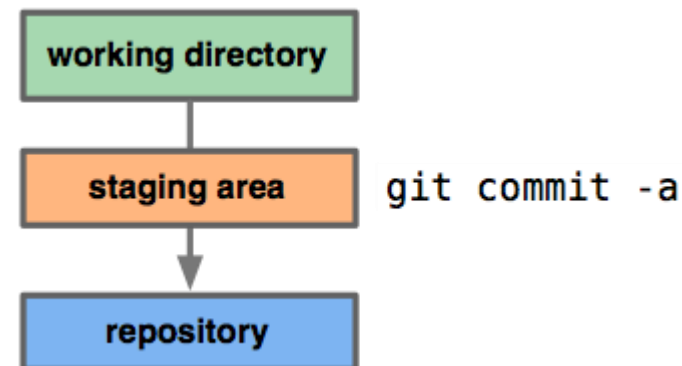
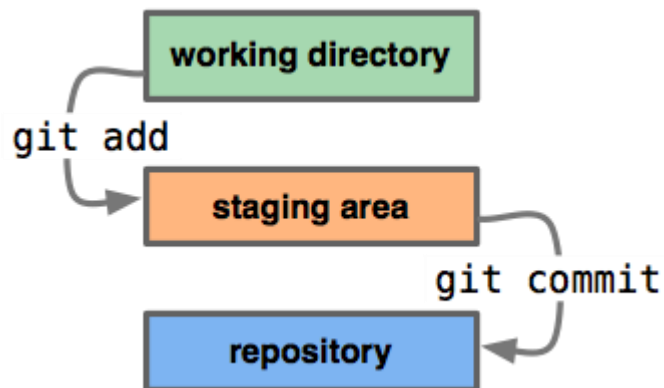
- La "staging area" (área de ensayo)
 - También denominada índice.
 - Es la zona donde se guardan los cambios provisionales.
 - NO es necesario añadir todos los archivos del área de trabajo a la staging área
- Para añadir un fichero al staging área:
 - `git add`
 - `git add .`
 - `git add <dir>`
- Para comprobar el estado del directorio
 - `git status`

Staging Area

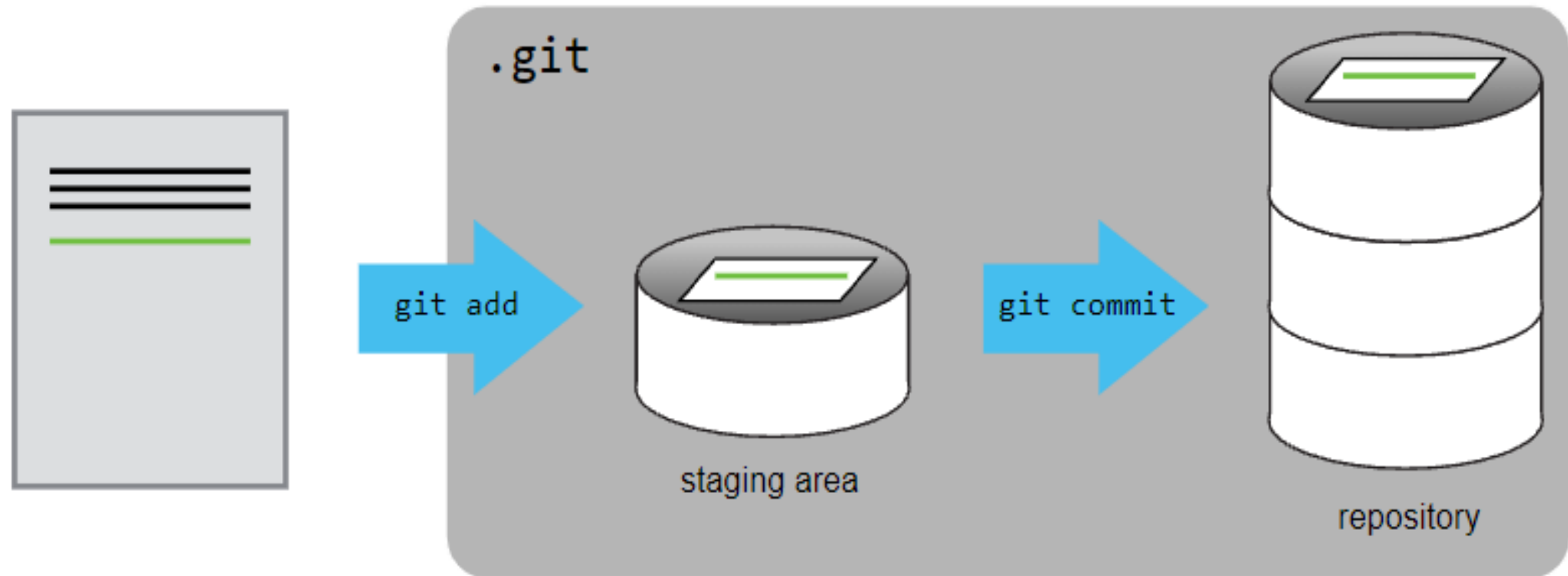
- Cambios
 - Si se cambia el fichero en el directorio local, entonces hay que volver a añadir el fichero al área de staging.
- Para mostrar los cambios a realizar en el staging area sin realizarlo:
 - `git add -n`.

Repositorio

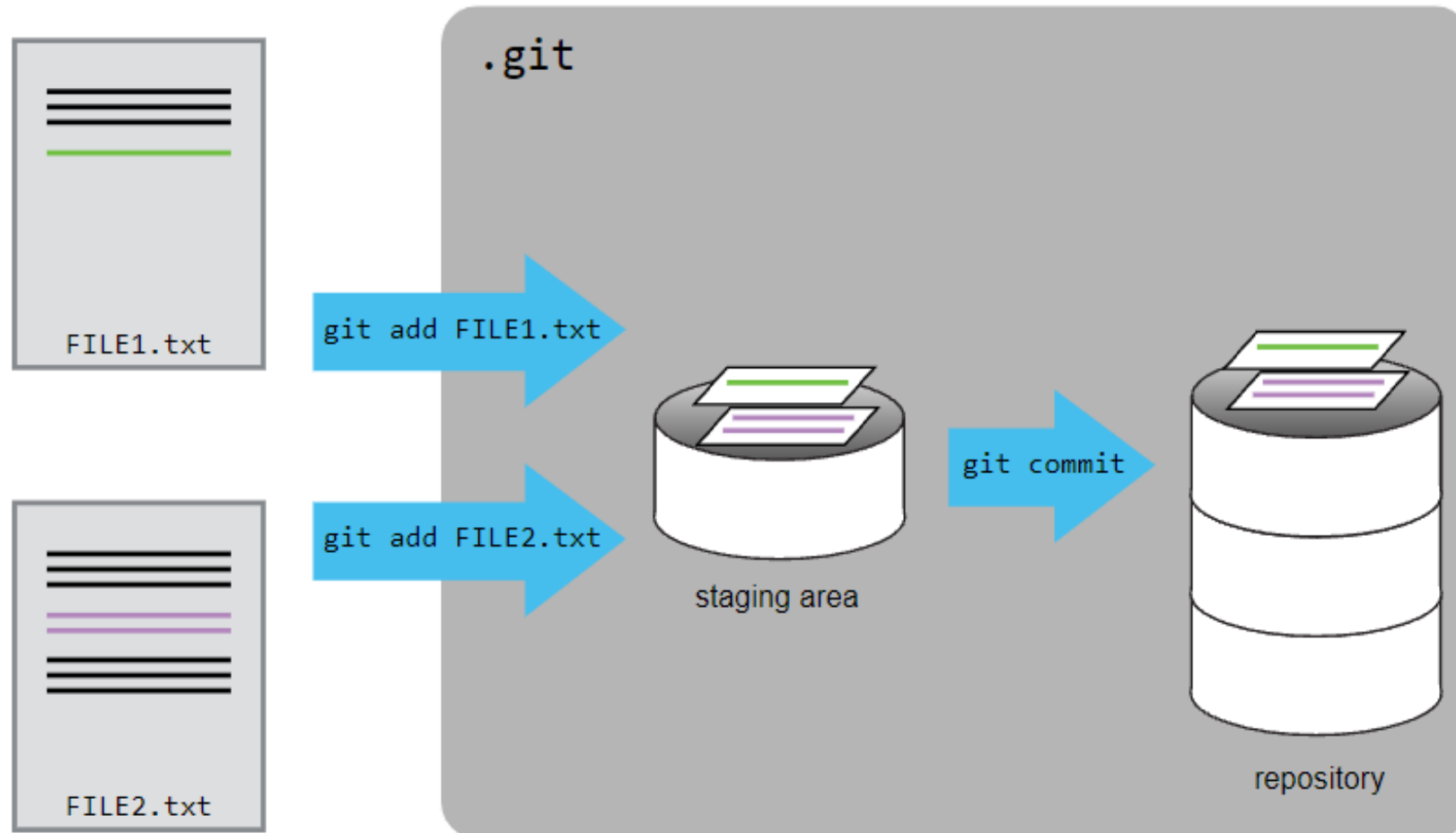
- Crea una instantánea del repositorio :
 - `git commit -m "un comentario"`
 - `git commit`
- Tiene en cuenta
 - El estado de la última instantánea realizada
 - El contenido de la staging area
- Observad que no se hace commit fichero a fichero.
- Debemos hacer commit frecuentemente. Con cambios pequeños.



Esquema de las áreas



Añadiendo varios ficheros



Borrar ficheros

- Para borrar un fichero hay dos alternativas:
 - Opción 1 (recomendada): `git rm`
 - Opción 2: `rm ; git rm [-f]`
- Elimina el archivo de la WC y anota en la staging area la eliminación.

Mover ficheros

- Para mover un fichero hay dos alternativas:
 - Opción 1 (recomendada): `git rm origen destino`
 - Opción 2: `mv origen destino; git rm origen; git add destino`
- Elimina el archivo de la WC y anota en la staging area la eliminación.

Otros comandos útiles

- Visualizar la historia de los commits
 - `git log [-p ] [-2]`
- Verificar el estado de la staging area
 - `git status` → Cambios pendientes de commit
 - `git diff` → Muestra los cambios de archivos modificados pero NO añadidos al staging area
 - `git diff --cached` → Muestra los cambios de archivos modificados que SÍ están añadidos al staging area
- Buscar el autor de un cambio
 - `git blame <file>`

Ignorando archivos

- Se puede indicar a git que ignore ciertos ficheros o carpetas
 - Crear el archivo .gitignore en la carpeta del proyecto
 - Es posible tener más archivos .gitignore en otras subcarpetas.
- Ejemplos de archivos .gitignore
 - <https://github.com/github/gitignore>
 - Gitlab también tiene templates que podemos usar

Ante un error

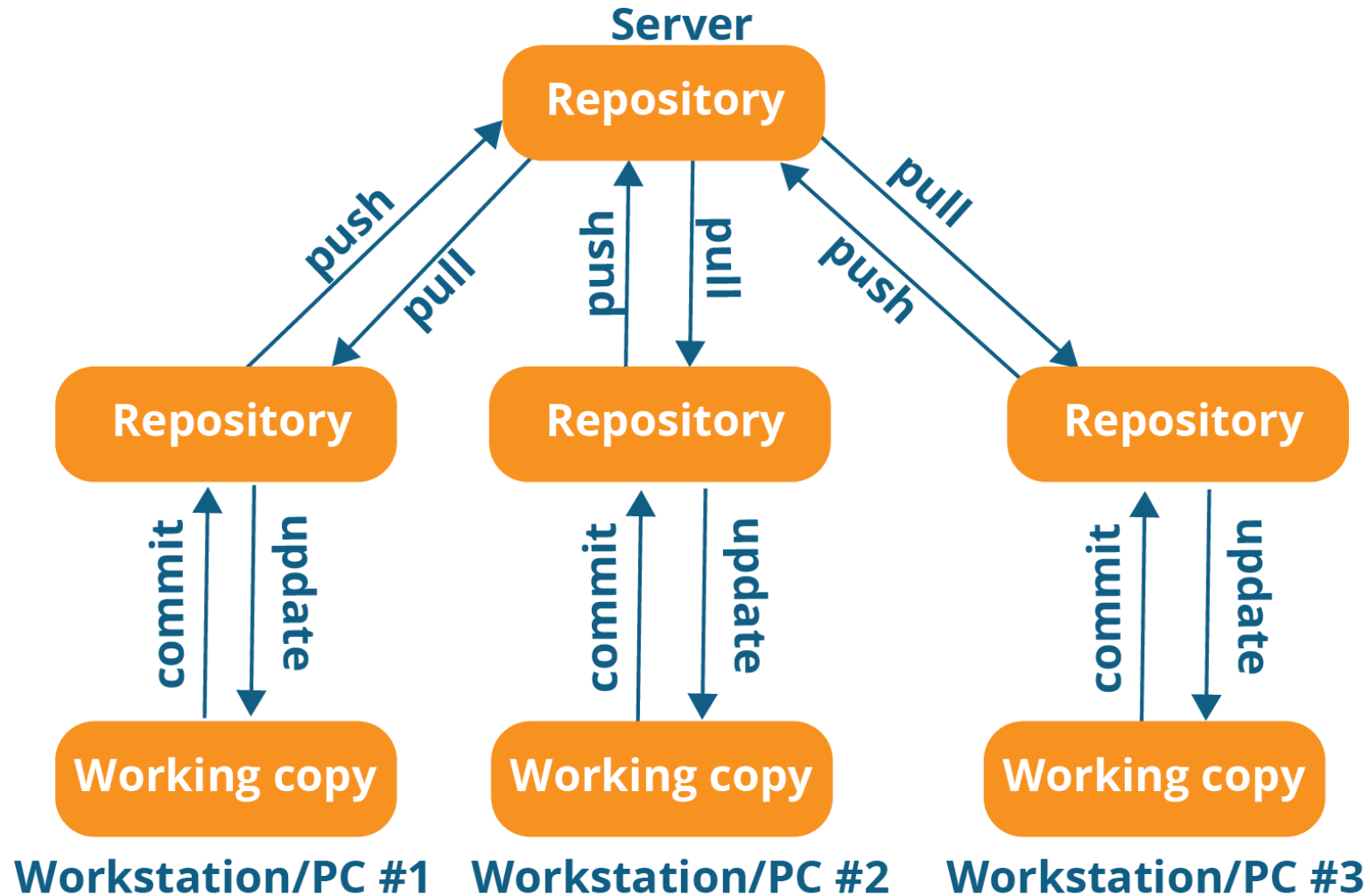
- Se desea cambiar el mensaje del último commit
 - `git commit --amend`
- Eliminar un archivo del staging area sin perder las modificaciones
 - Si el archivo es nuevo: `git rm --cached <archivo>`
 - Si el archivo ha sido modificado: `git reset HEAD <archivo>`
- Deshacer los cambios en la copia de trabajo y volver al archivo original desde la última instantánea
 - `git checkout -- <archivo>`

Repositorios remotos

- La colaboración entre desarrolladores se realiza a través de repositorios remotos
- Desde tu repositorio es posible acceder a otros repositorios para traerte cambios
- Modelo de repositorios públicos por desarrollador y un “*blessed*” repositorio
 - Crear un repositorio "maestro"
 - Crear un repositorio por desarrollador.

Repositorio remoto

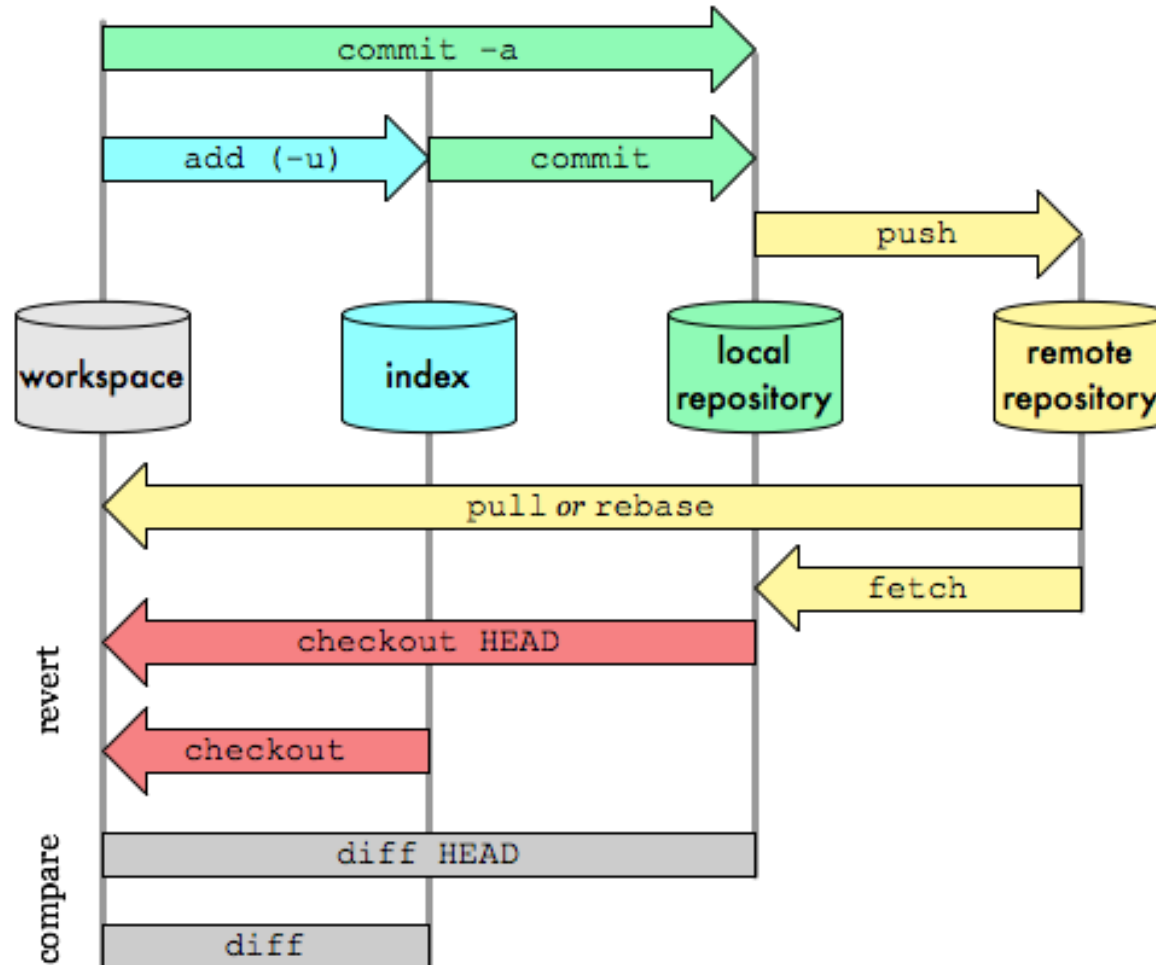
Distributed version control system



Flojo de la información

Git Data Transport Commands

<http://osteele.com>



Gitlab

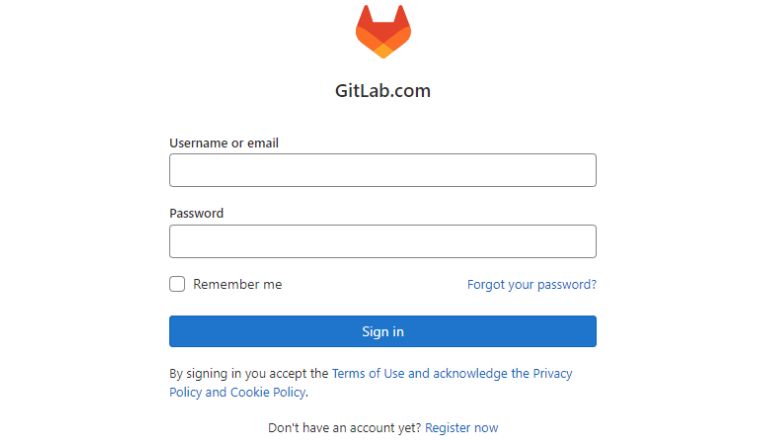
- Una aplicación web para gestionar un servidor git más otras funcionalidades añadidas (calidad, integración continua, etc.)

The screenshot displays the GitLab web interface for the 'GitLab' project. The left sidebar contains navigation links: Project information, Repository, Issues (39,264), Merge requests (1,208), Requirements, CI/CD, Deployments, Monitor, Packages & Registries, Analytics, and Snippets. The main content area shows the project details for 'GitLab' (Project ID: 278964), including statistics like 249,102 Commits, 8,152 Branches, 1,976 Tags, 8 GB Files, 66.4 TB Storage, and 115 Releases. A merge request is highlighted: 'Merge branch 'jv-gitaly-pack-objects-feature-flag' into 'master'' by Sean McGivern, authored 20 minutes ago. Below this, a table lists files and their commit history.

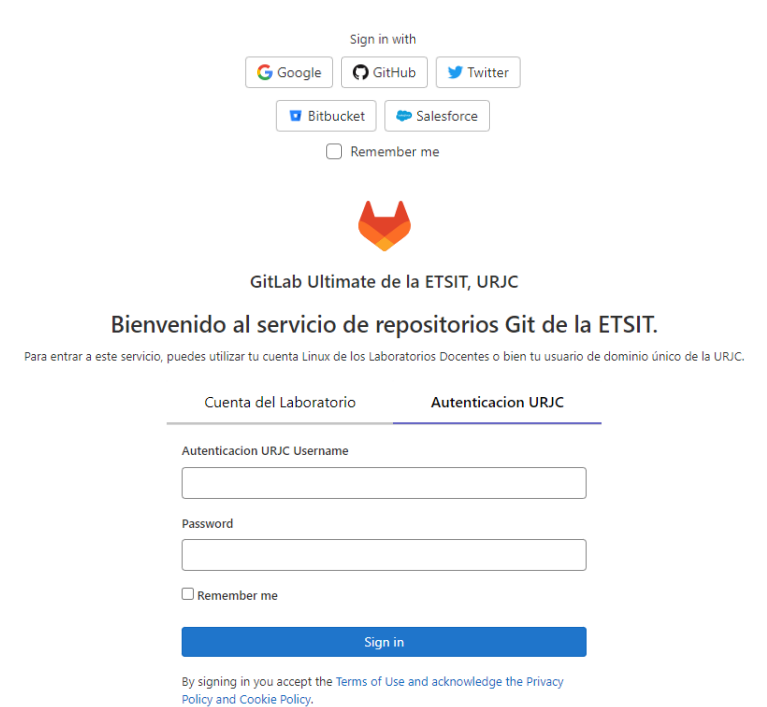
Name	Last commit	Last update
.github	Rename GitLab CE to FOSS in GitHub issue ...	1 year ago
.gitlab	Merge branch 'always-run-full-migration-sp...	2 hours ago
.theia	Revert "Move host authorization to to gitp...	10 months ago
app	Merge branch 'ph/324166/newReadyToMe...	1 hour ago
bin	Skip hooks when pushing security branche...	3 weeks ago
builds	Add missing builds/ folder to fix backup tests	5 years ago
changelogs	Remove wrapper methods from Gitlab::Dat...	4 weeks ago
config	Merge branch 'jv-gitaly-pack-objects-featur...	20 minutes ago
danger	Clarify the milestone requirements for sec...	4 days ago
data/whats_new	Create 14.2 In-app top 7 product highlights	1 day ago
db	Merge branch 'mc-remove-column-from-pr...	3 hours ago
doc	Merge branch 'docs-fix-dag-detail-2' into '...	32 minutes ago

Gitlab

- Puedes usar el servidor de gitlab público:
 - https://gitlab.com/users/sign_in
- O el que tenemos instalado en la universidad:
 - https://gitlab.etsit.urjc.es/users/sign_in



The screenshot shows the GitLab.com login page. At the top is the GitLab logo and the text "GitLab.com". Below this are two input fields: "Username or email" and "Password". There is a "Remember me" checkbox and a "Forgot your password?" link. A blue "Sign in" button is present. Below the button, it states: "By signing in you accept the Terms of Use and acknowledge the Privacy Policy and Cookie Policy." At the bottom, it says "Don't have an account yet? Register now".



The screenshot shows the GitLab Ultimate de la ETSIT, URJC login page. At the top is the GitLab logo and the text "GitLab Ultimate de la ETSIT, URJC". Below this is the heading "Bienvenido al servicio de repositorios Git de la ETSIT." and a note: "Para entrar a este servicio, puedes utilizar tu cuenta Linux de los Laboratorios Docentes o bien tu usuario de dominio único de la URJC." There are two tabs: "Cuenta del Laboratorio" and "Autenticacion URJC". The "Autenticacion URJC" tab is selected. Below it are two input fields: "Autenticacion URJC Username" and "Password". There is a "Remember me" checkbox and a blue "Sign in" button. At the bottom, it states: "By signing in you accept the Terms of Use and acknowledge the Privacy Policy and Cookie Policy."

Nuevo Proyecto



Projects

New project

Your projects 264

Starred projects 0

Explore projects

Explore topics

Pending deletion

Filter by name

Updated date

All Personal

Create new project

dated 1 month ago



Create blank project

Create a blank project to store your files, plan your work, and collaborate on code, among other things.



Create from template

Create a project pre-populated with the necessary files to get you started quickly.



Import project

Migrate your data from an external source like GitHub, Bitbucket, or another instance of GitLab.



Run CI/CD for external repository

Connect your external repository to GitLab CI/CD.

You can also create a project from the command line. [Show command](#)

Crear un proyecto



Create blank project

Create a blank project to store your files, plan your work, and collaborate on code, among other things.

New project > Create blank project

Project name

My awesome project

Project URL

https://gitlab.etsit.urjc.es/

Pick a group or namespace



/

Project slug

my-awesome-project

Want to organize several dependent projects under the same namespace? [Create a group.](#)

Visibility Level

☒ Private

Project access must be granted explicitly to each user. If this project is part of a group, access is granted to members of the group.

☐ Internal

The project can be accessed by any logged in user except external users.

☐ Public

The project can be accessed without any authentication.

Project Configuration

☒ Initialize repository with a README

Allows you to immediately clone this project's repository. Skip this if you plan to push up an existing repository.

☐ Enable Static Application Security Testing (SAST)

Analyze your source code for known security vulnerabilities. [Learn more.](#)

Create project

Cancel

Proyecto

The screenshot shows the GitLab interface for a new project named 'prueba'. The left sidebar contains navigation links: Project information, Repository, Issues (0), Merge requests (0), CI/CD, Security & Compliance, Deployments, Packages and registries, Infrastructure, Monitor, Analytics, Wiki, Snippets, and Settings. The main content area displays a success message: 'Project 'prueba' was successfully created.' Below this, the project name 'prueba' is shown with its ID (4386) and statistics: 1 Commit, 1 Branch, 0 Tags, and 61 KB Project Storage. An 'Auto DevOps' banner offers to automatically build, test, and deploy the application, with a link to the documentation and an 'Enable in settings' button. The 'main' branch is selected, and the 'prueba' directory is shown. A table lists the initial commit, 'Initial commit' by Agustin, with the commit hash 57c3e052. Below the table, there are buttons to add various files and configurations: README, LICENSE, CHANGELOG, CONTRIBUTING, Kubernetes cluster, and CI/CD. At the bottom, the file 'README.md' is listed, and the project name 'prueba' is displayed in a large font.

Search GitLab

prueba

Project information

Repository

Issues 0

Merge requests 0

CI/CD

Security & Compliance

Deployments

Packages and registries

Infrastructure

Monitor

Analytics

Wiki

Snippets

Settings

Agustin > prueba

Project 'prueba' was successfully created.

prueba

Project ID: 4386

1 Commit 1 Branch 0 Tags 61 KB Project Storage

Auto DevOps

It will automatically build, test, and deploy your application based on a predefined CI/CD configuration.

Learn more in the [Auto DevOps documentation](#)

Enable in settings

main prueba / +

Find file Web IDE Clone

Initial commit

Agustin authored just now

57c3e052

README Add LICENSE Add CHANGELOG Add CONTRIBUTING Add Kubernetes cluster Set up CI/CD

Configure Integrations

Name	Last commit	Last update
README.md	Initial commit	just now

README.md

prueba

git.ppt.pdf

TallerGitGitHub.pdf

Mostrar todo

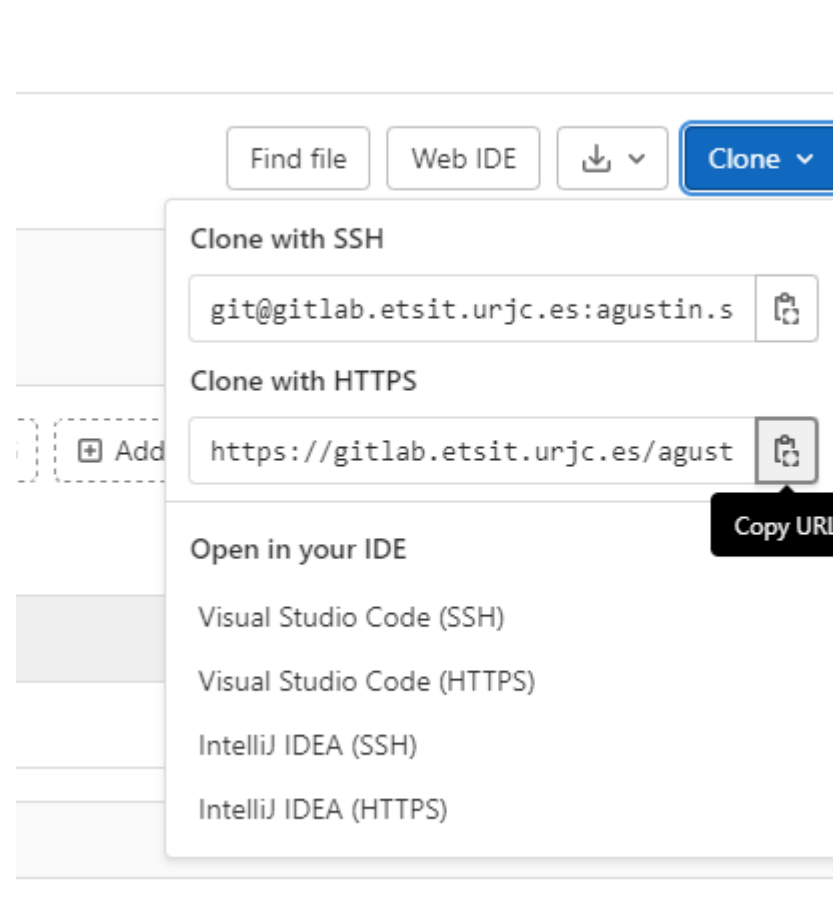
Clonar un proyecto

Clonar un proyecto a tu disco:

- `git clone url`

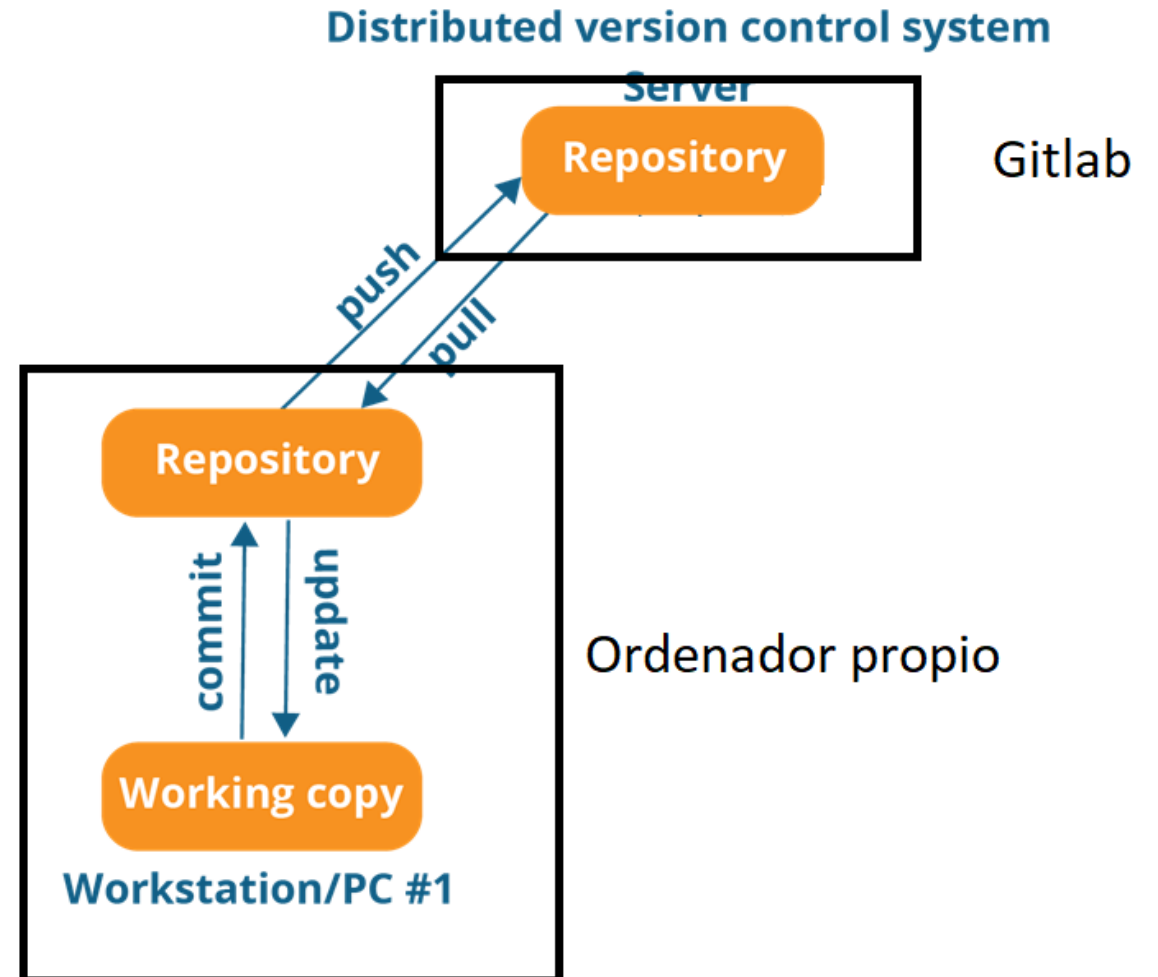
Conectar un proyecto ya existente:

- `cd existing_repo`
- `git remote add origin url`
- `git branch -M main`
- `git push -uf origin main`



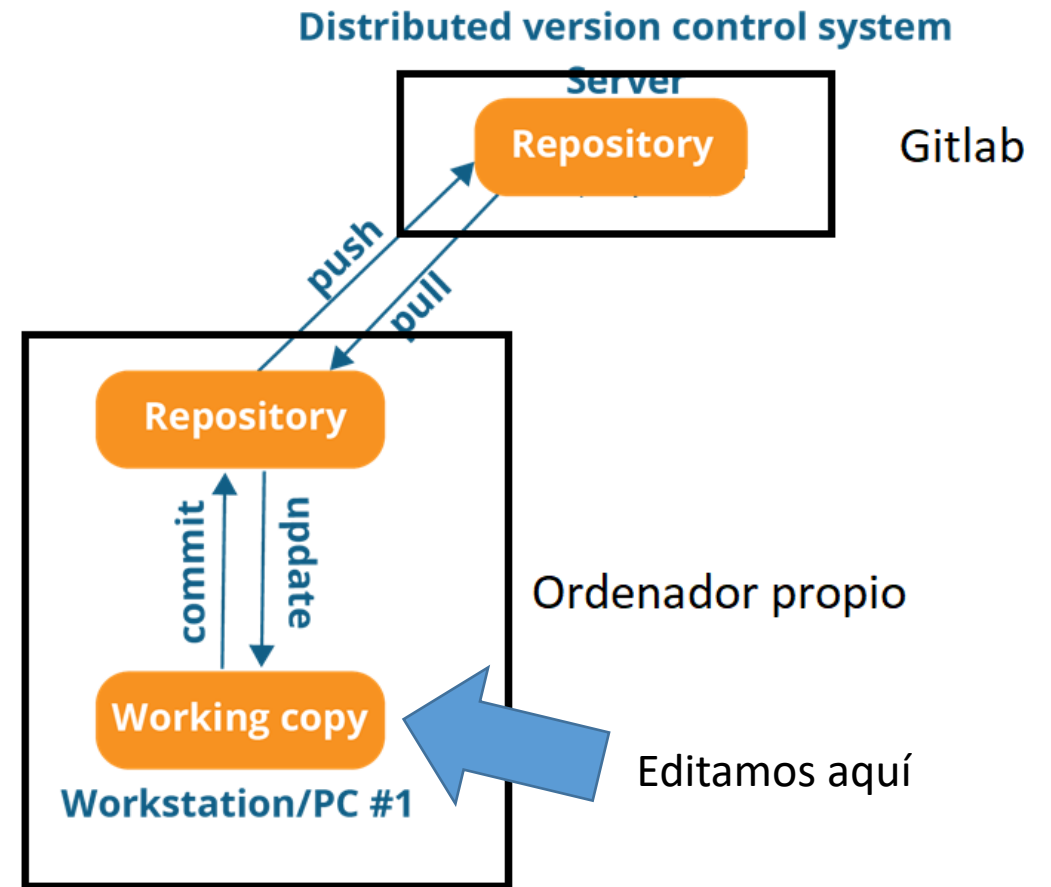
Estado actual

- En este punto tenemos dos repositorios:
 - Uno en local, situado en tu ordenador, dentro de la carpeta de trabajo
 - Otro remoto alojado en gitlab



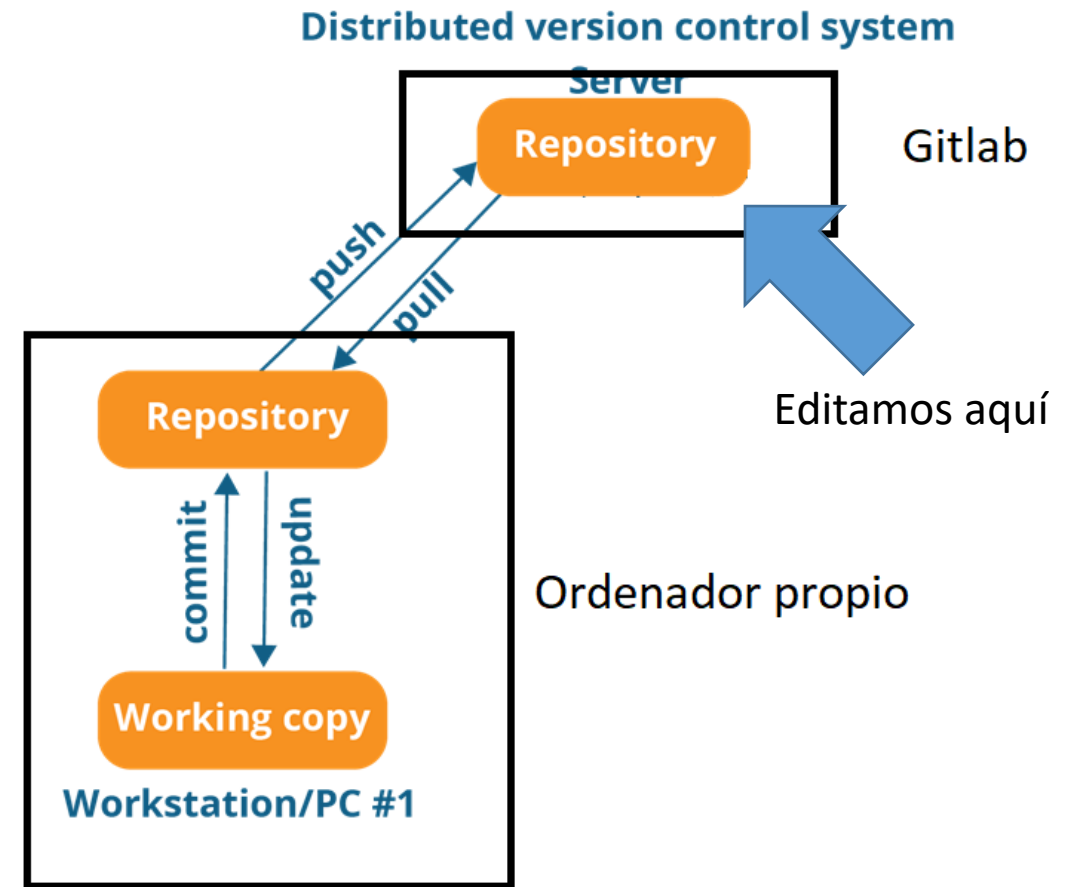
Flujo de trabajo habitual

- Editamos los ficheros
- Añadimos los ficheros cambiados al staging area
 - `git add .`
- Confirmamos el cambio
 - `git commit -m "un mensaje aclaratorio"`
- Sincronizamos los repositorios ("subimos" los cambios)
 - `git push`
- Repetimos...



Flujo de trabajo alternativo

- Editamos los ficheros en gitlab
- Sincronizamos los repositorios (“bajamos” los cambios)
 - git pull
- Repetimos...



Borrar un proyecto de gitlab

- Puedes borrar un proyecto desde

Settings->General->Advanced->Expand

The screenshot shows the GitLab web interface for a project named 'prueba'. The left sidebar contains a menu with options: Project information, Repository, Issues (0), Merge requests (0), CI/CD, Security & Compliance, Deployments, Packages and registries, Infrastructure, Monitor, Analytics, Wiki, Snippets, and Settings. The 'Settings' menu is expanded, showing sub-options: General, Integrations, Webhooks, Access Tokens, Repository, Merge requests, CI/CD, Packages and registries, Monitor, and Usage Quotas. The 'General' sub-option is selected. The main content area shows the 'Path' field with the value 'https://gitlab.etsit.urjc.es/agustin.santos/prueba' and a 'Change path' button. Below this is the 'Transfer project' section, which includes a warning about updating local repositories, a 'Select a new namespace' dropdown, and a 'Transfer project' button. At the bottom is the 'Delete this project' section, which includes a warning that the action is irreversible and a 'Delete project' button.

• You will need to update your local repositories to point to the new location.

Path

[Change path](#)

Transfer project

Transfer your project into another namespace. [Learn more.](#)

When you transfer your project to a group, you can easily manage multiple projects, view usage quota minutes, and users, and start a trial or upgrade to a paid tier.

Don't have a group? [Create one](#)

Things to be aware of before transferring:

- Be careful. Changing the project's namespace can have unintended side effects.
- You can only transfer the project to namespaces you manage.
- You will need to update your local repositories to point to the new location.
- Project visibility level will be changed to match namespace rules when transferring to a group.

Select a new namespace

[Transfer project](#)

Delete this project

This action deletes `agustin.santos/prueba` and everything this project contains. **There is no going back.**

[Delete project](#)

Ejercicios

- Trabajo en equipo con tus compañeros
- Elaborar la lista de comandos o de acciones que cumplan los objetivos de cada ejercicio.
- Publicar la “chuleta” en el foro del aula virtual
- Nota: algunas de los puntos implican varios comandos o acciones.

Ejercicios (I)

- Crea un proyecto en gitlab de la URJC
- Añade un fichero de Readme
- Clona ese proyecto en tu máquina
- Edita un fichero de Readme cambiando algo
- Añade ese fichero al repositorio local
- Sube ese cambio al repositorio remoto
- Comprueba en gitlab que se ha realizado el cambio

Ejercicios (II)

- Crea un proyecto en tu máquina
- Añade y edita un fichero de Readme
- Crea un proyecto con el mismo nombre en gitlab
- Conecta tu proyecto local con el que has creado en gitlab
- Edita el fichero de Readme desde gitlab
- Confirma el cambio
- Baja ese cambio al repositorio local
- Comprueba que se ha realizado el cambio en tu ordenador

Ejercicios (III)

- Usando alguno de los dos métodos anteriores, crea un proyecto en gitlab con todo el código que hemos realizado hasta la fecha
- Añade un Readme que explique el contenido del proyecto
- Clona el proyecto en otro ordenador (en el de casa o en los laboratorios).
 - Si no dispones de dos ordenadores puedes hacerlo en dos directorios diferentes
- Edita un fichero y propaga los cambios a todos los repositorios (tienes uno en el servidor y dos en local)

Ejercicios (IV)

- Queremos compartir el proyecto anterior con compañero
 - Concédale permisos para que pueda colaborar
- El compañero clona el proyecto en su ordenador y hace un cambio
- El compañero deberá subir sus cambios al repositorio remoto
- Debes sincronizar tu repositorio local y comprobar que el compañero ha realizado el cambio.

Comandos Git más utilizados

[git config](#)

[git log](#)

[git rebase](#)

[git merge](#)

[git fetch](#)

[git push](#)

[git add](#)

[git status](#)

[git reset](#)

[git checkout](#)

[git switch](#)

[git restore](#)

[git diff](#)

[git remote](#)

Resumen de comandos en:

<https://education.github.com/git-cheat-sheet-education.pdf>

Create a Repository

From scratch – Create a new local repository

```
$ git init [project name]
```

Download from an existing repository

```
$ git clone my_url
```

Observe a Repository

List new or modified files not yet committed

```
$ git status
```

Show the changes to files not yet staged

```
$ git diff
```

Show the changes to staged files

```
$ git diff --cached
```

Show all staged and unstaged file changes

```
$ git diff HEAD
```

Show the changes between two commit ids

```
$ git diff commit1 commit2
```

List the change dates and authors for a file

```
$ git blame [file]
```

Show the file changes for a commit id and/or file

```
$ git show [commit]:[file]
```

Working With Branches

List all local branches

```
$ git branch
```

List all branches, local and remote

```
$ git branch -av
```

Switch to a branch, my_branch, and update working directory

```
$ git checkout my_branch
```

Create a new branch called new_branch

```
$ git branch new_branch
```

Delete the branch called my_branch

```
$ git branch -d my_branch
```

Merge branch_a into branch_b

```
$ git checkout branch_b
```

```
$ git merge branch_a
```

Tag the current commit

```
$ git tag my_tag
```

Make a Change

Stages the file, ready for commit

```
$ git add [file]
```

Stage all changed files, ready for commit

```
$ git add .
```

Commit all staged files to versioned history

```
$ git commit -m "commit message"
```

Commit all your tracked files to versioned history

```
$ git commit -am "commit message"
```

Unstages file, keeping the file changes

```
$ git reset [file]
```

Revert everything to the last commit

```
$ git reset --hard
```

Synchroniz

Get the latest changes

```
$ git fetch
```

Fetch the latest changes

```
$ git pull
```

Fetch the latest changes

```
$ git pull --rebase
```

Push local changes

```
$ git push
```

Finally!

When in doubt, use

```
$ git [command]
```

Or visit training.github.com
training.

